Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ
к лабораторной работе №3
на тему:
«СТАНДАРТНЫЕ ТИПЫ ДАННЫХ, КОЛЛЕКЦИИ, ФУНКЦИИ, МОДУЛИ»
БГУИР 6-05-0612-02 78

Выполнил студент группы 453503
МОРОЗОВА Эвелина Сергеевна

_____
(дата, подпись студента)

Проверил доцент каф. Информатики
ЖВАКИНА Анна Васильевна

_____
(дата, подпись преподавателя)

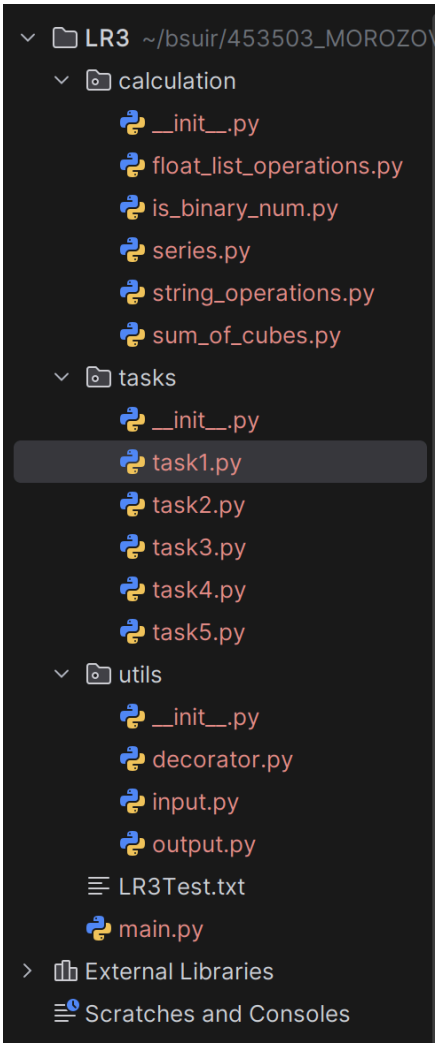
Минск 2026

1 ЦЕЛЬ

Освоить базовый синтаксис языка Python, приобрести навыки работы со стандартными типами данных, коллекциями, функциями, модулями и закрепить их на примере разработки интерактивных приложений.

2 ВЫПОЛНЕНИЕ ЗАДАНИЯ

В качестве среды разработки использовался PyCharm.



Для проверки ввода были написаны следующие функции:

```
"""
Module for input validation and sequence initialization.
```

```
Provides functions for safe user input with type checking and range
validation.
"""

def input_int(min_value: int, max_value: int, message: str) -> int:
    """Input integer number from min to max."""
    while True:
        try:
            value = int(input(message))
            if min_value <= value <= max_value:
                return value
            else:
                print("Invalid value. Please try again.")
                continue

        except ValueError:
            print("Invalid value. Please try again.")
            continue

def input_float(min_value: float, max_value: float, message: str) ->
float | None:
    """Input float number from min to max."""
    while True:
        try:
            value = float(input(message))
            if min_value < value < max_value:
                return value
            else:
                continue
        except ValueError:
            print("Invalid value. Please try again.")

def input_str(message: str) -> str | None:
    """Input string."""
    while True:
        try:
            value = input(message)
            if value != "":
                return value
            else:
                continue
        except ValueError:
            print("Invalid value. Please try again.")

def enter_sequence_float_with_required_num(seq: list, n: int, num:
float) -> list:
    """Enter a sequence with number 12 required."""
    if n == 1:
        return seq

    is_num_in_seq = False
    for i in range(n):
        member = input_float(-1000, 1000, "Enter a number: ")
```

```
        if member == num:
            is_num_in_seq = True
            break
        seq.append(member)

    if not is_num_in_seq:
              raise ValueError(f"Error: Number {num} is not in the
sequence.")
    return seq

def enter_sequence_float(seq: list, n: int) -> list:
    """Enter a sequence."""
    for i in range(n):
        member = input_float(-1000, 1000, "Enter a number: ")
        seq.append(member)
    return seq
```

Для вывода значений:

```
"""
This module implements console output functions.
"""
def print_menu():
       """Print a menu."""
       print("\nChoose an option:\n"
           "1. Task 1.\n"
           "2. Task 2.\n"
           "3. Task 3.\n"
           "4. Task 4.\n"
           "5. Task 5.\n"
           "6. Exit the program.\n")

def show_task1():
    """Show info about task1."""
       print("\nTask 1. arcsin calculation using series expansion.
Entering x, eps.\n")

def show_task2():
    """Show info about task2."""
      print("\nTask 2. Calculate sum of cubes in sequence. 12 - stop
marker.\n")

def show_task3():
    """Show info about task3."""
     print("\nTask 3. Program analyses text and defines if the enterrd
string is binary number.\n")

def show_task4():
    """Show info about task4."""
    print("\nTask 4. Find in text next options:\n"
          " a) determine amount of lowercase letters.\n"
```

```
            " b) find the last word containing the letter 'i' and its
number.\n"
               " c) display the string excluding words starting with
'i'.\n")

def show_task5():
    """Show info about task5."""
      print("\nTask 5. Enter the sequence. \nProgram finds amount of
elements that are bigger than C(input) and the product of the list
elements up to the maximum absolute value element.\n")

def print_table(x:float, n:int, f: float, mf: float, eps:float):
       """Print a table for task1."""
       print(f"{x} | {n} | {f} | {mf} | {eps}")

def print_sum(summ: int):
       """Print a sum for task2."""
       print(f"Sum of cubes: {summ}\n")

def print_choice_task2():
    """Print a choice task2."""
    print("\nChoose an option:\n"
          "1. Generate sequence.\n"
          "2. Enter by yourself.\n")
```

Меню программы:

```
"""
Lab 3.  Standard data types, collections, functions, modules.  Python
3.12.3.  Morozova E.S.  09.03.2026.
"""

from tasks.task1 import *
from tasks.task2 import *
from tasks.task3 import *
from tasks.task4 import *
from tasks.task5 import *
from utils.output import *
from utils.input import *

def main():
    """Execute the program."""
    while(True):
        print_menu()
        choice = input_int(1, 6, "Enter your choice: ")
        match choice:
            case 1:
                task1()
            case 2:
                task2()
            case 3:
                task3()
```

```
            case 4:
                task4()
            case 5:
                task5()
            case 6:
                break

if __name__ == "__main__":
    main()
```

Декоратор:

```
import time

def timer_decorator(func):
    """
    Decorator that measures and prints execution time of a function.

    Args:
        func: Function to be timed

    Returns:
        wrapper: Wrapped function with timing functionality

    Example:
        @timer_decorator
        def my_function():
            # function code
    """
    def wrapper(*args, **kwargs):
        """
          Wrapper function that adds timing before and after function
call.

        Args:
            *args: Variable length argument list
            **kwargs: Arbitrary keyword arguments

        Returns:
            Result of the wrapped function
        """
        start = time.time()
        res = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"Elapsed time: {elapsed:.5f}")
        return res
    return wrapper
```

Генератор

```
"""
Module for generator function and initialization.
"""

import random
from typing import Any, Generator


def num_generator_float(n: int):
    """Generator for integer numbers with number 12 required."""
    twelve_pos = random.randint(0, n-1)
    for i in range(n):
        if i == twelve_pos:
            break
        else:
            yield random.uniform(-1000, 1000)

def generate_sequence_float(seq: list, n: int) -> list:
    """Generate sequence using generator."""
    for number in num_generator_float(n):
        seq.append(number)
    return seq
```

1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд. Задать точность вычислений eps. Предусмотреть максимальное количество итераций, равное 500. Вывести количество членов ряда, необходимых для достижения указанной точности вычислений.

$$\arcsin x = \sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1} = x + \frac{x^3}{6} + \frac{3x^5}{40} + \ldots, |x| < 1$$

```
"""
Task 1: Calculate arcsin(x) using power series expansion.

Input: x (float in [-1.0, 1.0]), eps (float in [0.0, 1.0])
Output: Table showing iteration number, series value, math value, and
precision
Iterations limited to 500

Lab 3: Standard data types, collections, functions, modules
Python 3.12.3
Developer: Morozova E.S.
Date: 09.03.2026
"""
```

```
from utils.input import *
from calculation.series import *
from utils.decorator import *

@timer_decorator
def task1():
    """Execute task 1."""
    show_task1()
    x = input_float(-1.0, 1.0, "Enter x: ")
    eps = input_float(0.0, 1.0, "Enter eps: ")
    calculate_series(x, eps)

"""
Module for calculating arcsin(x) using power series expansion.
Provides function for series approximation.
"""
from math import asin, fabs
from utils.output import *

def calculate_series(x: float, eps: float) -> float:
    """
    Calculate arcsin(x) using power series expansion.
    Series: arcsin(x) = x + (1/6)x^3 + (3/40)x^5 + ...

    Args:
        x: Input value in range [-1.0, 1.0]
        eps: Precision (stopping criterion)

    Returns:
        float: arcsin(x) approximation using series

    Note:
        Maximum 500 iterations to prevent infinite loop
    """
    series = x
    a = x
    arcsin_value = asin(x)
    for i in range(500):
        print_table(x, i, series, arcsin_value, eps)
        if fabs(arcsin_value - series) < eps:
            break
        a *= (2*i + 1)**2 * x**2 / ((2*i + 2)*(2*i + 3))
        series += a
    return series
```

```
Task 1. arcsin calculation using series expansion. Entering x, eps.
Enter x: 0.3
Enter eps: 1e-10
0.3 | 0 | 0.3 | 0.3046926540153975 | 1e-10
0.3 | 1 | 0.3045 | 0.3046926540153975 | 1e-10
0.3 | 2 | 0.30468225 | 0.3046926540153975 | 1e-10
0.3 | 3 | 0.3046920133928571 | 0.3046926540153975 | 1e-10
0.3 | 4 | 0.3046926114006696 | 0.3046926540153975 | 1e-10
0.3 | 5 | 0.30469265103227827 | 0.3046926540153975 | 1e-10
0.3 | 6 | 0.3046926537988694 | 0.3046926540153975 | 1e-10
0.3 | 7 | 0.30469265399924966 | 0.3046926540153975 | 1e-10
Elapsed time: 7.93867
```

2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел. Организовать цикл, принимающий числа и суммирующий их кубы. Окончание цикла – ввод числа 12

```
"""
Task 2: Calculate sum of cubes with stop condition.

Program calculates sum of cubes of entered numbers.
Input stops if value equals 12.
User chooses between manual input and random generation.

Input: n (number of elements)
Output: Sum of cubes (excluding 12 itself)

Lab 3: Standard data types, collections, functions, modules
Python 3.12.3
Developer: Morozova E.S.
Date: 09.03.2026
"""
from calculation.sum_of_cubes import summary
from utils.input import input_int, generate_sequence, enter_sequence
from utils.output import print_choice_task2, show_task2


def task2():
    """Execute task 2."""
    show_task2()
    seq = []
    n = input_int(1, 100, "Enter n: ")
    print_choice_task2()
    choice = input_int(1, 2, "Enter choice: ")
    match choice:
        case 1:
            seq = generate_sequence(n)
```

```
            print(f"Generated sequence: {seq}")
        case 2:
            seq = enter_sequence(n)
            print(f"Entered sequence: {seq}")
    print(summary(seq))
"""
Module for calculating sum of cubes with special condition (stop on 12).
"""
from utils.input import input_int

def summary(seq:list) -> float:
    """
    Calculate sum of cubes until first 12.

    Args:
        seq: List of numbers

    Returns:
        Sum of cubes before first 12
    """
    summ = 0
    for num in seq:
        summ += num ** 3
    return summ
```

```
Task 2. Calculate sum of cubes in sequence. 12 - stop marker.
Enter n: 10

Choose an option:
1. Generate sequence.
2. Enter by yourself.

Enter choice: 1
Generated sequence: [-500, 690, -789, -537, 12, -143, 678, 506, -703, -640]
-442514222
```

3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры. Определить, является ли введенная с клавиатуры строка двоичным числом

```
"""
Task 3: Binary Number Validation.

Program checks if a user-entered string is a valid binary number.
A binary number can only contain digits 0 and 1.

Input: String from keyboard.
Output: Boolean result (True/False) indicating if string is binary.
```

```
Lab 3: Standard data types, collections, functions, modules.
Python 3.12.3.
Developer: Morozova E.S.
Date: 09.03.2026.
"""
from calculation.is_binary_num import is_binary
from utils.output import show_task3
from utils.input import input_str
def task3():
    """Execute task 3."""
    show_task3()
    str = input_str("Enter string: ").strip()
    print(f"String: {str} is binary num: {is_binary(str)}")

"""
Module for determination if the string is binary number.
"""
def is_binary(string: str) -> bool:
    """
    Check if string contains only binary digits (0 and 1).

    Args:
        string: Input string to check

    Returns:
        True if string consists only of '0' and '1', False otherwise
        Empty string returns False
    """
    for char in string:
        if char not in '01':
            return False
    return True
```

```
Task 3. Program analyses text and defines if the enterrd string is binary number.
Enter string: 01011011
String: 01011011 is binary num: True
```

4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы. а) определить количество строчных букв;
   б) найти последнее слово, содержащее букву 'i' и его номер;
   в) вывести строку, исключив из нее слова, начинающиеся с 'i'

```
"""
Task 4: Text Analysis.
```

```
Program analyzes given text and performs three operations:
a) Count lowercase letters.
b) Find the last word containing letter 'i' and its position.
c) Display text excluding words starting with 'i'.

Input: Predefined text string.
Output: Analysis results and modified text.

Lab 3: Standard data types, collections, functions, modules.
Python 3.12.3.
Developer: Morozova E.S..
Date: 09.03.2026.
"""
from calculation.string_operations import calculate_lowercase_letters,
find_last_word_ends_with_i, \
    delete_words_start_with_i
from utils.output import show_task4


def task4():
    """Execute task 4."""
    show_task4()
     s = "So she was considering spaghetti in her own mind, as well as she
could, for sushi the hot day made her feel very sleepy and stupid, whether
the pleasure of making a daisy-chain would be worth the trouble of getting up
and picking the daisies, when suddenly a White Rabbit kiwi with pink eyes ran
close by her.\n"
    print(s)
                    print(f"a)     Amount     of     lowercase     letters:
{calculate_lowercase_letters(s)}")
    last_word = find_last_word_ends_with_i(s)
      print(f"b) Last word that ends with 'i': {last_word[0]} at position:
{last_word[1]}")
              print(f"c)    String    without    words    start    with    'i':
{delete_words_start_with_i(s)}")

"""
Module for string operations in Task 4.
Provides functions for text analysis: counting lowercase letters,
finding words ending with 'i', and deleting words starting with 'i'.
"""
import string

def calculate_lowercase_letters(s: str) -> int:
    """
    Count the number of lowercase letters in a string.

    Args:
        s: Input string to analyze

    Returns:
        int: Total count of lowercase characters (a-z)
```

```
    """
    letters = list(s)
    lowercase_letters = [letter for letter in letters if letter.islower()]
    return len(lowercase_letters)

def find_last_word_ends_with_i(s: str) -> tuple:
    """
    Find the last word in text that ends with letter 'i' (case-insensitive).

    Args:
        s: Input text to search through

    Returns:
        tuple: (word, position) where:
            - word: the last word ending with 'i' (without punctuation)
            - position: index of this word in the original text
            Returns (None, -1) if no such word found
    """
    words_end_with_i = []
    words = s.split()
    positions = []
    for i, word in enumerate(words):
        clean_word = word.strip(string.punctuation)
        if clean_word and clean_word[-1].lower() == 'i':
            words_end_with_i.append(clean_word)
            positions.append(i)
    if words_end_with_i:
        return words_end_with_i[-1], positions[-1]
    else:
        return None, -1

def delete_words_start_with_i(s: str) -> str:
    """
    Delete all words that start with letter 'i' (case-insensitive).

    Args:
        s: Input text to process

    Returns:
        str: New string with all words starting with 'i' removed
             Words are joined with single spaces
    """
    words = s.split()
    new_s = []
    for word in words:
        clean_word = word.strip(string.punctuation)
        if clean_word and clean_word[0].lower() != 'i':
            new_s.append(word)
        elif not clean_word:
            new_s.append(word)
    return ' '.join(new_s)
```

```
Task 4. Find in text next options:
 a) determine amount of lowercase letters.
 b) find the last word containing the letter 'i' and its number.
 c) display the string excluding words starting with 'i'.

So she was considering spaghetti in her own mind, as well as she could, for sushi t
he hot day made her feel very sleepy and stupid, whether the pleasure of making a d
aisy-chain would be worth the trouble of getting up and picking the daisies, when s
uddenly a White Rabbit kiwi with pink eyes ran close by her.

a) Amount of lowercase letters: 243
b) Last word that ends with 'i': kiwi at position: 50
c) String without words start with 'i': So she was considering spaghetti her own mi
nd as well as she could for sushi the hot day made her feel very sleepy and stupid
whether the pleasure of making a daisy-chain would be worth the trouble of getting
up and picking the daisies when suddenly a White Rabbit kiwi with pink eyes ran clo
se by her
```

5. В соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции:

1) ввод элементов списка пользователем;
2) проверка корректности вводимых данных;
3) реализация основного задания с выводом результатов;
4) вывод списка на экран.

```
"""
Task 5: Float list operations.

Input: n, sequence of n floats, threshold c.
Output: count > c and product before max abs.

Lab 3.
Developer: Morozova E.S.
Date: 09.03.2026.
"""

from calculation.float_list_operations import
calculate_amount_of_nums_bigger_than_c,
calculate_product_of_nums
from utils.input import input_int, enter_sequence_float,
input_float
from utils.output import show_task5


def task5():
    """Execute task 5."""
    show_task5()
    n = input_int(1, 100, "Enter a n: ")
    seq = enter_sequence_float(n)
```

```
    c = input_float(-1000, 1000, "Enter a c: ")
        print(f"1. Amount of nums that are bigger than {c}:
{calculate_amount_of_nums_bigger_than_c(seq, c)}")
    print(f"2. Product of numbers before maximum absolute value:
{calculate_product_of_nums(seq)}")

"""
Module for float list operations in Task 5.
"""

def calculate_amount_of_nums_bigger_than_c(seq:list, c: float)
-> int:
    """
    Count the number of elements in sequence greater than given
value c.

    Args:
        seq: List of numbers (can be int or float)
        c: Threshold value to compare against

    Returns:
        int: Count of elements where element > c
    """
    return sum(1 for i in seq if i > c)

def calculate_product_of_nums(seq:list) -> float:
    """
    Multiply elements before max absolute value.

    Args:
        seq: List of numbers

    Returns:
         Product of elements before first occurrence of max abs
value
    """
    max_index = seq.index(max(seq, key=abs))
    product = 1
    for i in range(max_index):
        product *= seq[i]
    return product
```

```
Task 5. Enter the sequence.
Program finds amount of elements that are bigger than C(input) and the product of the list elements up to the maximum absolute value element.

Enter a n: 7
Enter a number: -4
Enter a number: 3
Enter a number: 2.4
Enter a number: -45
Enter a number: -34
Enter a number: 12
Enter a number: 55
Enter a c: 5
1. Amount of nums that are bigger than 5.0: 2
2. Product of numbers before maximum absolute value: -528767.9999999999
```